

Avaliação Geral e Segurança - OneClick Code

Data da avaliação: 05/06/2026

Escopo: revisão estática e somente-leitura do workspace local em `OneClick_Code`.

Observação: este relatório não substitui teste de invasão, revisão completa de código ou validação em ambiente de produção.

O projeto é um monorepo PNPM/Turbo com API NestJS/tRPC, web Next.js, Prisma/Postgres, Better Auth, módulos fiscais, certificados digitais, integrações Google/Stripe/SCI e aplicativos Electron.

A arquitetura tem boas bases, mas há riscos relevantes em rotas REST fora da camada tRPC, artefatos sensíveis no repositório/workspace, dependências vulneráveis e lacunas de autorização horizontal por empresa/tenant.

Achados Críticos

CRÍTICO Download de backup sem autenticação aparente

A rota `/api/backup/download/:filename` é registrada diretamente no Express em `apps/api/src/main.ts` e não valida sessão, permissão ou escopo do usuário.

Impacto: se a API estiver exposta e o nome de um arquivo for descoberto, um atacante pode baixar backups contendo dados sensíveis.

Recomendação: proteger a rota com autenticação, autorização de administrador, auditoria, tokens temporários e armazenamento fora do diretório da aplicação.

Referência: `apps/api/src/main.ts:64`

CRÍTICO Documentos fiscais expostos por endpoints REST

Vários controllers REST servem PDFs/XMLs sem autenticação local visível: CND, Sitfis, NFS-e e parte dos downloads DANFE.

Impacto: vazamento de documentos fiscais, XMLs, PDFs e dados de clientes se IDs forem descobertos por logs, links, histórico do navegador ou enumeração.

Recomendação: aplicar guard central em todos os controllers REST e checar ownership por `empresaId`, `clienteId` e permissões do módulo.

Referências: `cnd.controller.ts`, `sitfis.controller.ts`, `nfse-dist.controller.ts`, `danfe.controller.ts`

CRÍTICO Uploads sem autenticação e sem validação forte de conteúdo

O controller `/api/upload` permite upload genérico e upload de certificados sem autenticação aparente. Também serve arquivos por `/api/upload/:filename`.

Impacto: abuso de armazenamento, exposição indevida de anexos, upload de arquivos maliciosos com extensão permitida e possível mistura de arquivos entre usuários ou empresas.

Recomendação: exigir sessão, validar MIME e assinatura real do arquivo, criar escopo por empresa/usuário, restringir tipos permitidos por caso de uso e considerar varredura antivírus.

Referência: `apps/api/src/upload/upload.controller.ts`

CRÍTICO Segredos, dumps e certificado no workspace/repositório

Foram encontrados artefatos sensíveis no workspace e alguns versionados:

`certs/certificado.pfx`, `database-2026-04-11T19-17-47.sql`, `database-dump-2026-04-13.sql`, além de `.env`, backup de `.env`, dump de deploy e executável.

Impacto: vazamento de dados pessoais/fiscais, credenciais, chaves e certificados digitais.

Recomendação: remover do histórico Git quando aplicável, rotacionar chaves/senhas/certificados, armazenar dumps em cofre ou storage privado e reforçar `.gitignore`.

Autorização e Multiempresa

ALTO**Risco de autorização horizontal por empresa/tenant**

A camada tRPC possui `readProcedure`, `writeProcedure` e `deleteProcedure`, mas alguns services aceitam IDs e `empresaId` vindos do input. É necessário confirmar se todas as operações validam que o recurso pertence à empresa/tenant do usuário.

Impacto: usuário com permissão em um módulo pode conseguir alterar ou consultar dados de outra empresa caso conheça IDs ou envie `empresaId` indevido.

Recomendação: padronizar checagem de ownership nos services e criar testes de autorização horizontal para cada módulo crítico.

Referências: `apps/api/src/user/user.router.ts`, `apps/api/src/user/user.service.ts`

ALTO**Header `x-tenant-id` aceito diretamente**

O middleware de tenant aceita `x-tenant-id` vindo do cliente. O tRPC prioriza o tenant da sessão quando ela existe, mas rotas fora desse fluxo devem ser revisadas.

Recomendação: aceitar tenant apenas derivado da sessão/domínio confiável, ou validar explicitamente se o usuário pertence ao tenant informado.

Referência: `apps/api/src/tenant/tenant.middleware.ts`

Dependências

Foi executado `pnpm audit --prod`. Resultado: **30 vulnerabilidades**, sendo **12 high**, **16 moderate** e **2 low**.

- **Next.js:** múltiplas vulnerabilidades em `apps/web`, incluindo DoS, SSRF e bypass em App Router. O audit aponta correções em versões `>=15.5.16` e `>=15.5.18`.
- **xlsx 0.18.5:** prototype pollution e ReDoS. O audit não indica versão corrigida no pacote atual, o que sugere troca de biblioteca ou mitigação.
- **better-auth > kysely:** vulnerabilidade high em `kysely`, corrigida em `>=0.28.17`.
- **Transitivas via Puppeteer:** achados em `basic-ftp`, `ip-address`, `ws` e outros pacotes transitivos.

Pontos de Melhoria

- O build do Next ignora erros de TypeScript e ESLint em produção, em `apps/web/next.config.ts`. Isso pode permitir que bugs importantes cheguem ao deploy.
- CORS aceita origens LAN e qualquer origem em ambiente não produtivo. Em produção, usar allowlist estrita e revisar `TRUSTED_ORIGINS`.
- `docker-compose.yml` usa credenciais padrão `postgres/postgres` e expõe Postgres/Redis no host. Adequado para desenvolvimento, inadequado para produção.

- Há renderização de HTML com `dangerouslySetInnerHTML` em páginas públicas. O conteúdo deve ser sanitizado antes de persistir ou renderizar.
- Endpoints de atualização de apps Electron servem executáveis sem autenticação. Se forem públicos, garantir HTTPS, assinatura de artefatos, integridade e controle de origem.

Prioridade Recomendada

1. Proteger todos os controllers REST com autenticação e autorização, começando por backup, upload e documentos fiscais.
2. Remover do Git/workspace os artefatos sensíveis e rotacionar credenciais, chaves, senhas e certificados afetados.
3. Revisar autorização horizontal por `empresaId`, `tenantId`, `clienteId` e ownership nos services.
4. Atualizar dependências críticas, especialmente Next.js, better-auth/kysely e substituir ou mitigar `xlsx`.
5. Reativar falha de build para TypeScript/ESLint antes de tratar o deploy como estável.

Relatório gerado a partir de revisão estática local. Nenhum arquivo de código-fonte foi alterado durante a avaliação original.